

HTML y HTML5 para el desarrollo web

HTML no es código fuente: es un contrato semántico entre el autor del documento y el ecosistema de agentes que lo interpretan.

— Ian Hickson, editor de la especificación HTML5

Resultado de aprendizaje de la semana

Al concluir la semana 2, el estudiante construye documentos HTML5 estructuralmente correctos, semánticamente significativos y validables ante el W3C, integrando formularios con validación nativa, contenido multimedia accesible, APIs nativas del lenguaje y entendiendo el lugar de XML en el ecosistema de marcado.

2.1 Historia y evolución de HTML

El lenguaje HTML ha evolucionado durante tres décadas adaptándose a las demandas cambiantes de la web: del documento estático al *single-page application* interactivo. Conocer las etapas de esta evolución ayuda a entender por qué algunos tags existen, por qué otros se deprecaron y por qué HTML5 adoptó las formas actuales. La siguiente caja narra esa historia.

De SGML a HTML Living Standard

La historia de HTML no comienza con HTML sino con SGML (Standard Generalized Markup Language), estándar ISO 8879 publicado en 1986. SGML era un meta-lenguaje complejo que permitía definir lenguajes de marcado específicos para documentos de gobierno, ejército y empresa. Tim Berners-Lee, al inventar la web en 1989, tomó SGML como punto de partida pero buscó una versión radicalmente simplificada: HTML.

La primera descripción pública de HTML data de 1991 en una nota informal titulada *HTML Tags* que listaba apenas 18 etiquetas [6]. HTML 2.0 (1995, RFC 1866) fue el primer estándar formal, manteniendo la simplicidad pero añadiendo formularios. HTML 3.2 (W3C, 1997) incorporó tablas, applets y *scripting*. HTML 4.01 (1999) trajo la separación entre estructura y presentación al recomendar el uso de hojas de estilo CSS [50].

A finales de los noventa el W3C decidió reemplazar HTML por XHTML, una versión XML-estricta del lenguaje. Esta decisión, aunque técnicamente elegante, ignoró que la mayoría del HTML real en la web era *tag soup*: marcado mal formado pero que funcionaba en los navegadores. La presión por estandarizar las prácticas reales llevó a la formación, en 2004, del *Web Hypertext Application Technology Working Group* (WHATWG) por parte de Apple, Mozilla y Opera [25]. El WHATWG se desvinculó del proceso XHTML 2.0 y comenzó a desarrollar HTML5.

En 2014 el W3C publicó HTML5 como Recomendación oficial. En 2019 W3C y WHATWG firmaron un memorando reconociendo que el WHATWG sería el único editor del HTML Living Standard [62], abandonando la idea de versiones numeradas. Desde entonces, HTML evoluciona como *Living Standard*: cambios incrementales continuos

sin números de versión formales. Por convención y comodidad, seguimos hablando de `HTML5` para referirnos al HTML moderno.

La evolución de HTML refleja la transformación de la web misma: lo que comenzó como un sistema simple para compartir documentos científicos hipertextuales en el CERN se convirtió en la plataforma de software más ubicua del planeta. Cada versión documenta no solo adiciones técnicas sino también cambios profundos en cómo se entendía la naturaleza del documento web.

La tabla 2.1 resume los hitos. Conviene contextualizarla: las versiones *numeradas* de HTML (HTML 2.0, 3.2, 4.01) corresponden a una etapa en la que el lenguaje se actualizaba mediante *snapshots* formales del W3C. A partir de HTML5, y especialmente desde el memorando de 2019 entre WHATWG y W3C [62], el modelo cambió a *Living Standard*: la especificación se actualiza continuamente sin saltos de versión. Esto refleja la madurez del lenguaje: no necesita grandes revisiones disruptivas, solo evoluciones incrementales.

Tabla 2.1: Hitos de la evolución de HTML.

Versión	Año	Aportes principales
HTML <code>!DOCTYPE</code>	1991	18 etiquetas iniciales (<code><a></code> , <code><p></code> , <code><h1></code> ...).
HTML 2.0	1995	Primera estandarización formal (RFC 1866); formularios.
HTML 3.2	1997	Tablas, applets, fuentes, alineación.
HTML 4.01	1999	Marcos, scripts, separación de presentación (CSS).
XHTML 1.0	2000	Reformulación XML de HTML 4.01.
XHTML 1.1	2001	Modularización; estricta sintaxis XML.
HTML5 borrador	2008	WHATWG inicia HTML5; nuevos elementos semánticos.
HTML5 W3C Rec.	2014	Recomendación oficial; <code><video></code> , <code><canvas></code> , APIs.
HTML 5.1	2016	Mejoras menores (<code><picture></code> , <code><details></code>).
HTML 5.2	2017	<code><dialog></code> , modal nativo.
HTML 5.3	—	Cancelado en favor del Living Standard.
HTML Living Standard	desde 2019	Estado actual; cambios continuos por WHATWG.

La trayectoria muestra tres etapas claramente diferenciadas: (a) la etapa fundacional (1991–1999) donde HTML pasa de prototipo a estándar formal; (b) el periodo de transición XML (2000–2008) en el que el W3C intentó migrar HTML al ecosistema XML con XHTML 1.0 y 2.0, esfuerzo que finalmente fracasó porque la web real era demasiado tolerante a errores [26]; y (c) la era moderna de HTML5 (desde 2008) en la que el lenguaje recupera su pragmatismo original mientras incorpora capacidades multimedia, APIs avanzadas y semántica rica.

2.2 Fundamentos de HTML

HTML (*HyperText Markup Language*) es un lenguaje de marcado declarativo cuyo propósito es describir la estructura semántica de un documento hipertextual. A diferencia de los lenguajes procedurales o imperativos, HTML no *hace* nada: describe *qué es* cada parte del contenido y deja al navegador la responsabilidad de renderizarlo. Como argumentan Duckett [15] y Robbins [53], esta naturaleza declarativa es lo que ha permitido que HTML sobreviva 35 años casi sin cambios sintácticos: agentes muy diversos (navegadores, lectores de pantalla, motores de búsqueda, crawlers, extractores de datos, asistentes de IA) pueden interpretar el mismo documento de maneras distintas pero coherentes.

Sus tres pilares son:

- **Etiquetas (tags).** Construyen la estructura. Vienen en pares de apertura y cierre: `<p>texto</p>`. Algunas son *void* (no contienen contenido): `<img>`, `<br>`, `<input>`.
- **Atributos.** Aportan información adicional sobre los elementos: `class`, `id`, `lang`, `aria-*`. Se escriben dentro de la etiqueta de apertura como pares `nombre="valor"`.
- **Contenido.** El texto y los elementos hijos que se anidan dentro de las etiquetas.

2.2.1 Estructura mínima de un documento HTML5 válido

Todo documento HTML5 válido debe contener cinco elementos imprescindibles: (i) la declaración `<!DOCTYPE html>` en la primera línea; (ii) un elemento raíz `<html>` con atributo `lang`; (iii) una sección `<head>` con metadatos (al menos `<meta`

`charset` y `<title>`); (iv) una sección `<body>` con el contenido visible; y (v) idealmente, una estructura semántica que separe cabecera, navegación, contenido principal y pie.

El siguiente código presenta una página HTML5 completa que ilustra el uso de **todos** los tags principales: tanto los nuevos tags semánticos de HTML5 (`<header>`, `<nav>`, `<main>`, `<section>`, `<article>`, `<aside>`, `<footer>`, `<figure>`, `<time>`) como los tags clásicos que siguen siendo fundamentales (`<div>`, `<table>`, `<form>`, `<ul>`, `<a>`, `<img>`). Este código sirve como **patrón canónico** de referencia para construir cualquier página web.

Ejemplo 2.1 — Documento HTML5 completo con tags clásicos y semánticos

Enunciado. Construir un documento HTML5 mínimo pero completo que incluya todos los tags semánticos esenciales (`<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>`) y los tags clásicos de organización (`<div>`, `<table>`). Servirá como página de referencia para el resto de los ejercicios.

```

1 <!DOCTYPE html>
2 <html lang="es-EC">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <meta name="description" content="Pagina demo con todos los tags
   clave">
7   <meta name="author" content="UTEQ - Aplicaciones Web">
8   <title>Pagina completa con tags clave - UTEQ</title>
9 </head>
10 <body>
11
12 <!-- Tags HTML5 semanticos: estructura -->
13 <header>
14   <h1>Universidad Tecnica Estatal de Quevedo</h1>
15   <p>La primera universidad agropecuaria del Ecuador</p>
16   <nav aria-label="Menu principal">
17     <ul>
18       <li><a href="#inicio">Inicio</a></li>
19       <li><a href="#carreras">Carreras</a></li>
20       <li><a href="#contacto">Contacto</a></li>
21     </ul>
22   </nav>
23 </header>
24
25 <main>
26   <article id="inicio">
27     <h2>Bienvenidos al sitio web</h2>
28     <p>Publicado el <time datetime="2026-05-04">4 de mayo
29     de 2026</time>.</p>
30     <figure>
31       
34       <figcaption>Campus central UTEQ - Quevedo</figcaption>
35     </figure>
36   </article>
37
38   <section id="carreras">
39     <h2>Carreras de la Facultad</h2>

```

```

40
41 <!-- Tag clasico: tabla para datos tabulares -->
42 <table>
43   <caption>Carreras de Ciencias de la Computacion</caption>
44   <thead>
45     <tr><th scope="col">Carrera</th>
46       <th scope="col">Duracion</th>
47       <th scope="col">Titulo</th></tr>
48   </thead>
49   <tbody>
50     <tr><td>Ingenieria de Software</td>
51       <td>9 semestres</td>
52       <td>Ing. de Software</td></tr>
53     <tr><td>Ingenieria en Sistemas</td>
54       <td>9 semestres</td>
55       <td>Ing. de Sistemas</td></tr>
56   </tbody>
57 </table>
58 </section>
59
60 <section id="contacto">
61   <h2>Contactenos</h2>
62
63   <!-- Tag clasico: formulario -->
64   <form action="/api/contacto" method="POST">
65     <!-- Tag clasico: div como contenedor generico -->
66     <div>
67       <label for="nombre">Nombre:</label>
68       <input id="nombre" name="nombre" type="text" required>
69     </div>
70     <div>
71       <label for="email">Correo:</label>
72       <input id="email" name="email" type="email" required>
73     </div>
74     <div>
75       <label for="msg">Mensaje:</label>
76       <textarea id="msg" name="msg" rows="4"></textarea>
77     </div>
78     <button type="submit">Enviar</button>
79   </form>
80 </section>
81
82 <aside aria-label="Datos adicionales">
83   <h2>Datos rapidos</h2>
84   <ul>
85     <li>Fundacion: <strong>1984</strong></li>
86     <li>Estudiantes: <strong>13.000+</strong></li>
87     <li>Programas: <strong>22</strong></li>
88   </ul>
89 </aside>
90 </main>

```

```

91
92     <footer>
93         <small>&copy; 2026 UTEQ. Todos los derechos reservados.</small>
94         <address>
95             Av. Quito km 1.5 via a Santo Domingo<br>
96             Telefono: <a href="tel:+59353702220">(+593) 5 3702-220</a>
97         </address>
98     </footer>
99
100 </body>
101 </html>

```

Listing 2.1: pagina-completa.html

Explicación de las decisiones tomadas:

- `<header>`, `<nav>`, `<main>`, `<article>`, `<section>`, `<aside>`, `<footer>`: la estructura semántica de HTML5 reemplaza el uso clásico de `<div class="header">`, `<div id="content">`.
- `<table>`: usada *solo* para datos tabulares, con `<caption>`, `<thead>`, `<tbody>` y atributo `scope` en los `<th>`, conforme a las prácticas accesibles [53].
- `<form>`, `<input>`, `<label>`: cada `<input>` tiene su `<label for>` asociado (criterio WCAG 1.3.1).
- `<div>`: usado solo cuando no hay un elemento semántico apropiado, como contenedor neutro para cada grupo `<label>+<input>`.
- `<time datetime>`: fecha en formato ISO 8601 legible por máquinas.

2.2.2 Las cinco categorías de contenido en HTML5

HTML5 abandona la dicotomía clásica *block-level* / *inline* (propia de HTML 4) y la sustituye por las **categorías de contenido**, que se solapan y permiten razonar con mayor precisión sobre qué puede contener qué [26]. Este cambio conceptual es significativo: antes, los desarrolladores aprendían reglas mecánicas (¿nun `<p>` puede ir dentro de un `<a>`¿); ahora aprenden categorías (¿nun elemento de *phrasing content* no admite *flow content*¿), lo que generaliza mucho mejor.

A continuación se presentan las cinco categorías principales con ejemplos de uso en una página web real:

1. **Flow content** (la mayoría del contenido del cuerpo): texto, párrafos, listas, encabezados, imágenes, formularios, secciones. Casi todo lo que puede ir en `<body>` es flow content. *Ejemplo de uso*: dentro de un `<section>` podemos poner `<h2>`, `<p>`, `<ul>`, `<table>`, `<form>`: todos son flow content.
2. **Phrasing content** (texto y sus marcas en línea): `<em>`, `<strong>`, `<a>`, `<span>`, `<code>`, `<time>`, `<mark>`. Es un subconjunto de *flow*. *Ejemplo de uso*: dentro de un párrafo podemos escribir `<p>El libro <em>1984</em> fue publicado en <time datetime="1949">1949</time>.</p>`.
3. **Embedded content** (contenido externo o interactivo incrustado): `<img>`, `<video>`, `<audio>`, `<canvas>`, `<iframe>`, `<object>`, `<svg>`. *Ejemplo de uso*: dentro de un `<figure>` incrustamos `<img>` y agregamos `<figcaption>`.
4. **Interactive content** (elementos con los que el usuario interactúa): `<a>` con `href`, `<button>`, `<input>` (excepto `type="hidden"`), `<select>`, `<textarea>`, `<details>`, `<label>`. *Ejemplo de uso*: dentro de un `<form>` ponemos varios `<input>` interactivos.
5. **Sectioning content** (secciones del documento): `<article>`, `<section>`, `<nav>`, `<aside>`. Cada uno define un nuevo ícontexto de outlinez que afecta la jerarquía de encabezados. *Ejemplo de uso*: la página de la sección anterior tiene un `<article>` (sectioning content) que contiene un `<h2>` de bienvenida.

Existen además categorías menos comunes: *metadata content* (`<meta>`, `<link>`, `<style>`, `<title>`), *heading content* (`<h1>`-`<h6>`), *form-associated content* (`<input>`, `<button>`, etc.). Un mismo elemento puede pertenecer a varias categorías simultáneamente: por ejemplo, `<a>` es flow, phrasing e interactive.

2.2.3 Modelo de contenido y reglas de anidamiento

Cada elemento HTML define qué puede contener y qué no, mediante un **modelo de contenido** declarado en la especificación. Estas reglas no son arbitrarias: derivan de la semántica del elemento y de limitaciones técnicas históricas. Comprenderlas evita errores sutiles que pasan desapercibidos a primera vista pero rompen la accesibilidad o el SEO.

Reglas fundamentales con ejemplos:

1. **<p> solo admite phrasing content.** Por eso un párrafo no puede contener un `<div>` o un `<table>`. **Anti-ejemplo (inválido) (Listado 2.2):**

```
<p>Aqui hay una tabla:
  <table><tr><td>X</td></tr></table>
</p>
```

Listing 2.2: Ejemplo de HTML

Correcto: cerrar el `<p>` antes de la tabla.

2. **<a> acepta cualquier contenido excepto interactive content.** Por eso un enlace no puede contener otro enlace, ni un `<button>`, ni un `<input>`. **Anti-ejemplo (inválido) (Listado 2.3):**

```
<a href="/x"><button>Click</button></a>
```

Listing 2.3: Ejemplo de HTML

Correcto: usar solo `<a>` con texto y CSS para darle apariencia de botón, o solo `<button>` con `onclick` que redirija.

3. **<button> no admite interactive content.** Por eso no se puede tener un `<a>` dentro de un `<button>`.
4. ** y solo aceptan (y opcionalmente <script>/<template>).** **Anti-ejemplo (inválido) (Listado 2.4):**

```
<ul>
  <h2>Items</h2>
  <li>Uno</li>
  <li>Dos</li>
</ul>
```

Listing 2.4: Ejemplo de HTML

Correcto: el encabezado va *antes* de la `<ul>`.

5. **<table> requiere estructura específica:** solo admite `<caption>`, `<colgroup>`, `<thead>`, `<tbody>`, `<tfoot>`, `<tr>` (este último solo sin `<thead>/<tbody>`). Las filas `<tr>` solo admiten `<td>` y `<th>`.
6. **<form> no puede contener otro <form>.** Pero sí puede tener `<input form="otroForm">` que asocia un input con un form externo (técnica avanzada).

¿Por qué el navegador es indulgente? Los navegadores intentan recuperarse de HTML inválido por compatibilidad histórica. Sin embargo:

- Los *validadores* (W3C) reportan errores.
- Los *lectores de pantalla* pueden anunciar contenido incoherente.
- Los *motores de SEO* penalizan estructuras inválidas.
- Las *herramientas automatizadas* (extracción de datos, asistentes de IA) producen resultados impredecibles.

Por estas razones, todo HTML profesional debe pasar el W3C Markup Validator [64] sin errores antes de ir a producción.

2.3 Elementos semánticos de HTML5 en profundidad

Antes de HTML5 la mayoría de la maquetación se hacía con `<div>` genéricos diferenciados por clases CSS. Este enfoque *semánticamente ciego* tenía varios problemas documentados [32]: los motores de búsqueda no entendían la estructura del contenido, los lectores de pantalla no podían ofrecer navegación rápida entre regiones, y el código se volvía ilegible para humanos. HTML5 introdujo un vocabulario semántico explícito que mejora la accesibilidad, el SEO, la mantenibilidad y la interoperabilidad con herramientas de *reader mode*, extracción automatizada y asistentes de IA.

2.3.1 Catálogo de elementos semánticos

La tabla 2.2 sintetiza los elementos semánticos esenciales de HTML5 con su uso correcto e incorrecto. La razón para incluir ambas columnas es pedagógica: en los entregables del curso, los errores más frecuentes consisten en usar elementos semánticos para fines genéricos (por ejemplo, `<section>` como contenedor visual sin encabezado), lo que invalida su semántica.

Tabla 2.2: Elementos semánticos esenciales de HTML5 con su uso correcto e incorrecto.

Elemento	Uso correcto	Uso incorrecto
<code><header></code>	Cabecera de página o sección	Como contenedor genérico
<code><nav></code>	Bloque de navegación principal	Para listas de enlaces secundarios
<code><main></code>	Contenido principal único	En múltiples instancias
<code><article></code>	Contenido autónomo y reutilizable	Para cualquier párrafo
<code><section></code>	Agrupación temática con encabezado	Sin <code><h2>/<h3></code> dentro
<code><aside></code>	Contenido relacionado pero secundario	Para barras laterales decorativas
<code><footer></code>	Pie de página o sección	Para contenido principal
<code><figure></code>	Imagen, código o tabla autocontenida	Para cualquier imagen
<code><figcaption></code>	Pie de figura	Fuera de <code><figure></code>
<code><time></code>	Fecha u hora marcada para máquinas	Sin atributo <code>datetime</code>
<code><mark></code>	Texto destacado contextualmente	Como sinónimo de <code></code>
<code><details>+<summary></code>	Bloque colapsable	Para acordeones complejos
<code><dialog></code>	Modal nativo	Para tooltips simples
<code><address></code>	Datos de contacto	Para direcciones postales arbitrarias

Para cada elemento, la regla mnemotécnica es: *¿si puedo explicar en una frase qué significa este bloque, debo poder elegir el elemento semántico apropiado; si solo puedo describirlo por su apariencia visual, probablemente debo usar un `<div>`?*

2.3.2 ¿`<article>` o `<section>`? Guía decisional con ejemplo real

La confusión más común es cuándo usar `<article>` y cuándo `<section>`. Ambos son *sectioning content*, ambos crean un nuevo contexto de outline, pero tienen semánticas distintas [32, 53].

Regla heurística canónica:

- `<article>`: si el bloque de contenido podría *publicarse de manera independiente* en otro contexto y seguir teniendo sentido, es un artículo. La prueba RSS: ¿tendría sentido incluir este bloque en un feed RSS aislado de su contexto?

Ejemplos canónicos: una entrada de blog, una noticia, un comentario en un foro, una tarjeta de producto, un *tweet*, una reseña.

- `<section>`: si el bloque es una agrupación temática que solo tiene sentido como parte del documento mayor.

Ejemplos: la sección 'Estudios' de un currículum, el capítulo de un libro, la sección 'Características' de una landing page.

Ejemplo real con código completo: blog con múltiples entradas.

Ejemplo 2.2 — Uso correcto de `<article>` y `<section>`

```

1 <main>
2   <h1>Blog de Aplicaciones Web - UTEQ</h1>
3
```

```

4  <!-- ARTICLE: entrada que puede publicarse en RSS independiente -->
5  <article>
6      <header>
7          <h2>Por que HTML5 cambio el desarrollo web</h2>
8          <p>Por <a href="/autor/jdoe" rel="author">Juan Doe</a>
9              el <time datetime="2026-05-04">4 mayo 2026</time></p>
10         </header>
11
12         <p>Introduccion al articulo...</p>
13
14         <!-- SECTION dentro de article: agrupacion tematica interna -->
15         <section>
16             <h3>Elementos semanticos nuevos</h3>
17             <p>HTML5 introdujo header, nav, main...</p>
18         </section>
19
20         <section>
21             <h3>APIs multimedia nativas</h3>
22             <p>Las etiquetas video y audio reemplazaron Flash...</p>
23         </section>
24
25         <footer>
26             <p>Etiquetas:
27                 <a href="/tag/html5">HTML5</a>,
28                 <a href="/tag/web">Web</a>
29             </p>
30         </footer>
31     </article>
32
33     <!-- Otro ARTICLE independiente -->
34     <article>
35         <header>
36             <h2>CSS Grid: el sistema de maquetacion definitivo</h2>
37             <p>Por <a href="/autor/asmith">Ana Smith</a>
38                 el <time datetime="2026-04-28">28 abril 2026</time></p>
39         </header>
40         <p>CSS Grid revoluciono la maquetacion web...</p>
41     </article>
42
43     <!-- SECTION: agrupacion que NO tendria sentido fuera del blog -->
44     <section aria-label="Articulos relacionados">
45         <h2>Tambien te puede interesar</h2>
46         <ul>
47             <li><a href="/post/1">Una guia rapida de Flexbox</a></li>
48             <li><a href="/post/2">JavaScript moderno en 2026</a></li>
49         </ul>
50     </section>
51 </main>

```

Listing 2.5: blog.html (extracto significativo)

Razonamiento de las decisiones:

- Cada entrada del blog es `<article>`: si se extrae y publica en otro sitio o en un feed, tiene sentido por sí misma.
- Dentro de cada artículo, las `<section>` agrupan subtemas pero *no* podrían publicarse independientemente: `ñAPIs multimedia nativas` sin el contexto del artículo carece de significado.
- `ñArtículos relacionados` es una `<section>` porque es una agrupación temática (lista de enlaces) que pertenece a la página del blog pero no es *contenido* autónomo.

2.4 Formularios HTML5 y validación nativa

Los formularios constituyen el principal mecanismo de entrada de datos del usuario hacia el servidor. HTML5 amplió drásticamente las capacidades de validación nativa, reduciendo la dependencia de JavaScript para tareas básicas y mejorando significativamente la experiencia del usuario en dispositivos móviles [41]. Con HTML5 podemos confiar en el navegador para verificar formato de email, rangos numéricos, longitudes mínimas y máximas, y patrones complejos mediante expresiones regulares.

Es importante destacar: **la validación HTML5 es para usabilidad, no para seguridad**. Un atacante puede saltarse fácilmente las validaciones del cliente; por eso la validación robusta debe duplicarse en el servidor (semanas 5–9). Sin embargo, la validación HTML5 mejora la experiencia ofreciendo retroalimentación inmediata al usuario *antes* de enviar el formulario.

2.4.1 Tipos de input modernos

A los tipos clásicos de input que existían desde HTML 2.0 (`text`, `password`, `checkbox`, `radio`, `submit`, `hidden`, `file`), HTML5 añadió **trece tipos modernos** que tres ventajas operacionales fundamentales: *(i)* validación automática del formato (correo, URL, número), *(ii)* interfaces de entrada especializadas (calendarios, selectores de color, sliders) y *(iii)* en móviles, invocan automáticamente el teclado virtual más apropiado para el tipo de dato (numérico para `tel`, con `@` para `email`, etc.).

La tabla 2.3 cataloga estos tipos modernos junto con su uso recomendado y la validación nativa que cada uno implementa. Conviene memorizar esta tabla: usar el tipo correcto de input es una de las maneras más simples y efectivas de mejorar la usabilidad de un formulario.

Tabla 2.3: Tipos de `<input>` de HTML5 y su validación nativa.

Tipo	Uso	Validación nativa
<code>email</code>	Correo electrónico	Formato <code>x@y.z</code>
<code>tel</code>	Teléfono	No valida formato (varía por país); usar <code>pattern</code>
<code>url</code>	URL	Formato URL absoluto
<code>number</code>	Número	Solo dígitos; <code>min/max/step</code>
<code>range</code>	Rango (slider)	Numérico; sin teclado de entrada
<code>date</code>	Fecha	Calendario nativo
<code>time</code>	Hora	Selector horario
<code>datetime-local</code>	Fecha y hora local	Calendario + reloj
<code>month</code>	Año-mes	Selector de mes
<code>week</code>	Semana del año	Selector de semana
<code>color</code>	Color	Selector de color
<code>search</code>	Búsqueda	Cosmética: borrar con X

En móviles, además, cada tipo de input invoca el teclado virtual adecuado: `email` muestra el carácter `@`; `number` y `tel` muestran el teclado numérico; `url` muestra `.com` y `/`. Esto reduce drásticamente los errores de tipeo en pantallas pequeñas.

2.4.2 Atributos de validación

Los siguientes atributos permiten especificar restricciones que el navegador valida automáticamente antes de enviar el formulario:

- `required`: el campo no puede enviarse vacío.
- `minlength` / `maxlength`: rango de longitud de texto.

- **min / max**: rango numérico o temporal.
- **pattern**: expresión regular (sintaxis JavaScript) que debe cumplir el valor.
- **step**: incremento permitido en numéricos y fechas (**step**="0.01" para dinero, **step**="60" para minutos).
- **autocomplete**: directrices al navegador (**on**, **off**, **name**, **email**, **tel**, **cc-number**, **street-address**, **country**, **bday**...).
- **novalidate** (en **<form>**): deshabilita la validación nativa.
- **multiple**: permite múltiples valores (**<input type="email" multiple>**).
- **accept**: tipos MIME permitidos en **<input type="file">**.

Ejemplo de formulario completo con validación nativa:

Ejemplo 2.3 — Formulario de inscripción a curso con validación HTML5 completa

Enunciado. Construir un formulario de inscripción a curso con validación HTML5 nativa (**required**, **type**, **pattern**, **min/max**) que rechace datos inválidos antes de enviarlos al servidor.

```

1 <form action="/api/inscripcion" method="POST">
2   <fieldset>
3     <legend>Datos personales</legend>
4
5     <label for="cedula">Cedula (10 digitos):</label>
6     <input id="cedula" name="cedula" type="text"
7       pattern="[0-9]{10}" required
8       autocomplete="off"
9       aria-describedby="ayuda-cedula">
10    <small id="ayuda-cedula">Sin guiones ni espacios</small>
11
12    <label for="nombre">Nombre completo:</label>
13    <input id="nombre" name="nombre" type="text"
14      minlength="3" maxlength="80" required
15      autocomplete="name">
16
17    <label for="email">Correo institucional:</label>
18    <input id="email" name="email" type="email"
19      pattern=".+@uteq\.edu\.ec" required
20      autocomplete="email">
21
22    <label for="tel">Telefono celular:</label>
23    <input id="tel" name="tel" type="tel"
24      pattern="09[0-9]{8}" required
25      autocomplete="tel">
26
27    <label for="fnac">Fecha de nacimiento:</label>
28    <input id="fnac" name="fnac" type="date"
29      min="1980-01-01" max="2010-12-31" required
30      autocomplete="bday">
31  </fieldset>
32
33  <fieldset>
34    <legend>Curso</legend>
35

```

```

36     <label for="nivel">Nivel academico (1-10):</label>
37     <input id="nivel" name="nivel" type="number"
38           min="1" max="10" step="1" required>
39
40     <label for="prom">Promedio acumulado:</label>
41     <input id="prom" name="prom" type="number"
42           min="6.0" max="10.0" step="0.01" required>
43
44     <label for="hrs">Horas semanales de estudio:</label>
45     <input id="hrs" name="hrs" type="range"
46           min="0" max="40" step="1" value="20">
47
48     <label for="carrera">Carrera:</label>
49     <select id="carrera" name="carrera" required>
50       <option value="">-- Seleccione --</option>
51       <option value="SW">Ingenieria de Software</option>
52       <option value="SE">Ingenieria de Sistemas</option>
53     </select>
54 </fieldset>
55
56 <button type="submit">Inscribirme</button>
57 <button type="reset">Limpiar</button>
58 </form>

```

Listing 2.6: inscripcion.html (formulario funcional)

Validaciones automáticas que aplica el navegador:

- Si la cédula no tiene exactamente 10 dígitos: error nativo.
- Si el email no termina en @uteq.edu.ec: error nativo.
- Si el teléfono no comienza con 09: error nativo.
- Si el promedio está fuera de rango 6.0–10.0: error nativo.
- Si el campo carrera no se selecciona: error nativo.

Todo esto sin escribir una sola línea de JavaScript.

Nota didáctica: las pseudo-classes CSS para dar retroalimentación visual al estado de validación (:valid, :invalid, :required, :user-invalid) se estudiarán en detalle en la semana 3, cuando dispongamos del marco conceptual completo de CSS. Por ahora, basta con saber que existen y que podemos darles estilo al campo según esté correctamente lleno o no.

2.5 Multimedia y APIs de HTML5

HTML5 incorporó al estándar nativo cinco APIs de gran importancia práctica que reemplazaron tecnologías propietarias como Flash y Silverlight [34]. Su importancia no es solo técnica: políticamente representó la victoria de los estándares abiertos sobre los plugins propietarios. Apple inició la batalla en 2010 cuando Steve Jobs publicó la carta abierta *íThoughts on Flash* [28] declarando que iOS nunca soportaría Flash. La transición a HTML5 nativo se completó alrededor de 2020, cuando Adobe discontinuó oficialmente Flash Player.

Las cinco APIs nativas son:

1. <video> y <audio>: reproducción multimedia nativa.
2. <canvas>: gráficos 2D y 3D (con WebGL).
3. **Geolocation API**: posición geográfica del usuario con consentimiento.
4. **Web Storage**: localStorage y sessionStorage.

5. **Drag-and-Drop API:** arrastrar y soltar entre componentes.

2.5.1 La etiqueta <video> con accesibilidad completa

La etiqueta <video> es uno de los elementos más complejos de HTML5 porque integra múltiples preocupaciones: formato del video, controles de reproducción, accesibilidad para usuarios sordos (subtítulos), accesibilidad para usuarios ciegos (audiodescripciones), optimización de carga (preload, poster) y compatibilidad con navegadores distintos (múltiples <source>).

Para construir un video accesible profesional necesitamos conocer los siguientes atributos y elementos asociados:

- **controls:** muestra los controles nativos del navegador (reproducir, pausa, volumen, pantalla completa).
- **preload:** indica al navegador cuánto descargar antes de reproducir. Valores: **none** (nada), **metadata** (solo cabeceras: duración y dimensiones), **auto** (todo).
- **poster:** imagen que se muestra antes de iniciar la reproducción (mejora la percepción de carga).
- **autoplay:** reproducción automática (la mayoría de navegadores la bloquean si el video tiene audio, por buenas razones).
- **loop:** reproducción en bucle.
- **muted:** silenciado por defecto (necesario para autoplay).
- Múltiples <source>: el navegador usa el primer formato que reconoce. Buena práctica: WebM primero (Chrome/Firefox), MP4 después (Safari/Edge).
- <track>: pistas de subtítulos, descripciones, metadatos. Sus tipos (**kind**):
 - **captions:** subtítulos para usuarios sordos en el idioma del video (criterio WCAG 1.2.2).
 - **subtitles:** subtítulos traducidos a otro idioma.
 - **descriptions:** descripciones de audio para usuarios ciegos (criterio WCAG 1.2.5).
 - **chapters:** marcadores de capítulos.
 - **metadata:** datos para scripts.

Ejemplo 2.4 — Video accesible profesional

Enunciado. Mostrar la etiqueta <video> con todos los atributos de accesibilidad necesarios en producción: **controls**, **preload**, **poster**, pistas de subtítulos con <track> y fuentes alternativas con <source>.

```

1 <figure>
2   <video controls width="640"
3     preload="metadata"
4     poster="poster-uteq.jpg"
5     crossorigin="anonymous">
6     <source src="bienvenida.webm" type="video/webm">
7     <source src="bienvenida.mp4" type="video/mp4">
8     <track kind="captions" src="bienvenida.es.vtt"
9       srclang="es" label="Español" default>
10    <track kind="captions" src="bienvenida.en.vtt"
11      srclang="en" label="English">
12    <track kind="descriptions" src="bienvenida.desc.vtt"
13      srclang="es" label="Audiodescripcion">
14    <p>Su navegador no soporta video HTML5.
15      <a href="bienvenida.mp4">Descargar el video</a>.</p>

```

```

16 </video>
17 <figcaption>Mensaje de bienvenida del Rector --- Mayo
    2026</figcaption>
18 </figure>

```

Listing 2.7: Ejemplo de HTML

Aspectos clave de este código:

- Múltiples `<source>`: el navegador elige el primero soportado.
- Subtítulos en español y en inglés (`default` marca el predeterminado).
- `<track kind="descriptions">`: audiodescripciones para usuarios ciegos.
- Texto fallback dentro de `<video>`: navegadores muy antiguos.
- Envuelto en `<figure>`+`<figcaption>` para semántica correcta.

Formato del archivo .vtt (WebVTT) para subtítulos (Listado 2.8):

```

1 WEBVTT
2
3 00:00:00.000 --> 00:00:04.000
4 Bienvenidos a la Universidad Tecnica Estatal de Quevedo.
5
6 00:00:04.500 --> 00:00:08.000
7 Soy el Rector y les hablo desde nuestro campus central.

```

Listing 2.8: bienvenida.es.vtt

2.5.2 El elemento `<picture>` para imágenes responsivas

El elemento `<picture>` resuelve un problema concreto: servir *la imagen correcta para cada dispositivo*. Antes de `<picture>`, la web servía la misma imagen a todos los dispositivos, lo que penalizaba a usuarios móviles con redes lentas: una imagen optimizada para desktop (1200 px de ancho) cargaba también en un teléfono que solo necesitaba 400 px, gastando datos innecesarios y prolongando tiempos de carga.

`<picture>` permite al navegador elegir la imagen óptima según: (i) el tamaño del viewport (media queries), (ii) la densidad de pixels (Retina, etc.) y (iii) el formato soportado (WebP es 25–35 % más pequeño que JPEG, AVIF es 50 % más pequeño aún, pero no todos los navegadores los soportan).

Ejemplo 2.5 — Imagen optimizada para múltiples dispositivos

Enunciado. Implementar una imagen responsive con `<picture>` y `<source>` para servir el formato y tamaño más apropiados según el dispositivo y soporte del navegador.

```

1 <picture>
2 <!-- Para pantallas grandes con AVIF soportado -->
3 <source media="(min-width: 1200px)"
4       srcset="banner-grande.avif"
5       type="image/avif">
6 <source media="(min-width: 1200px)"
7       srcset="banner-grande.webp"
8       type="image/webp">
9 <source media="(min-width: 1200px)"
10      srcset="banner-grande.jpg"
11      type="image/jpeg">
12
13 <!-- Para tablets -->

```

```
14 <source media="(min-width: 600px)"
15       srcset="banner-medio.webp"
16       type="image/webp">
17
18 <!-- Fallback para todos -->
19 
23 </picture>
```

Listing 2.9: Ejemplo de HTML

Beneficios:

- El navegador descarga *solo* la versión adecuada al viewport y al formato soportado.
- AVIF/WebP reducen el peso 25–50 % respecto a JPEG con calidad equivalente.
- `loading="lazy"` difiere la descarga hasta que la imagen está cerca del viewport.
- Las dimensiones explícitas (`width`, `height`) evitan el *layout shift* durante la carga.

2.6 XML y familias de lenguajes de marcado

HTML pertenece a una familia mayor de lenguajes de marcado que descienden de SGML. XML (*eXtensible Markup Language*) es el otro gran descendiente de SGML, estandarizado por el W3C en 1998 [65]. La diferencia fundamental: HTML tiene un vocabulario fijo (los tags están predefinidos); XML es *extensible*, el autor define su propio vocabulario.

Esta extensibilidad permitió que XML se convirtiera en el formato de intercambio dominante en sistemas empresariales durante los 2000s. Aunque desde 2015 ha sido desplazado parcialmente por JSON en APIs REST modernas, XML sigue siendo esencial en muchos contextos: configuración de aplicaciones Java (Maven, Spring), servicios web SOAP, sindicación de contenidos, formatos de documentos (DOCX, ODT, EPUB son ZIP de archivos XML).

2.6.1 Subdialectos de XML relevantes en aplicaciones web

XML como tecnología generó múltiples subdialectos especializados que siguen vigentes en distintas áreas. Conocer cuáles están en uso en aplicaciones web reales permite reconocerlos cuando aparecen en bibliotecas, configuraciones o servicios externos. La lista siguiente enumera los más relevantes en 2026.

- **XHTML**: HTML reformulado como XML estricto. Hoy relegado pero existe en casos específicos (correos electrónicos, algunos sistemas legados).
- **SVG** (Scalable Vector Graphics): gráficos vectoriales, nativos en HTML5. Iconos, gráficos, diagramas que escalan sin pérdida de calidad.
- **MathML**: notación matemática. Usada en sitios académicos y educativos.
- **RSS** y **Atom**: sindicación de contenido.
- **SOAP**: servicios web XML (semana 15).
- **XSLT**: transformaciones XML a otros formatos.
- **DocBook**: documentación técnica.
- **XAML**: interfaces de WPF y WinUI.

2.6.2 Formatos de intercambio de datos: XML, JSON, YAML, TOML y otros

XML y JSON no son los únicos formatos de intercambio de datos relevantes en 2026. Para que el desarrollador profesional pueda elegir bien, conviene conocer el panorama completo. Cada formato tiene fortalezas y casos de uso óptimos.

Tabla 2.4: Formatos de intercambio de datos modernos.

Formato	Características	Casos de uso 2026
XML	Verboso; etiquetas anidadas con atributos; validación con XSD; comentarios soportados.	Maven <code>pom.xml</code> , SOAP, SVG, configuración legada (Spring XML).
JSON	Sintaxis tipo objeto JS; tipos básicos; sin comentarios; soporte nativo en JavaScript.	APIs REST, <code>package.json</code> , NoSQL (MongoDB).
YAML	Indentación significativa; muy legible; tipos inferidos; soporta comentarios.	Docker Compose, Kubernetes, GitHub Actions, Spring Boot <code>application.yml</code> .
TOML	Sintaxis INI mejorada; tipado explícito; secciones; pensado para configuración.	Rust (<code>Cargo.toml</code>), Python (<code>pyproject.toml</code>).
INI	Formato <code>clave=valor</code> con secciones; muy simple.	Configuración legada en Windows, PHP <code>php.ini</code> .
CSV	Tabular plano; valores separados por comas.	Exportación a Excel, ETL básico.
Protobuf	Binario; eficiente; requiere esquema <code>.proto</code> ; soporte multi-lenguaje.	gRPC, comunicación servicio-servicio de alto rendimiento.
MessagePack	Binario tipo JSON; más compacto.	Comunicación IoT, caches binarias.
Avro	Binario; esquema JSON embebido.	Apache Kafka, big data, streaming.
Parquet	Columnar binario; comprimido.	Análisis de datos a escala (Spark, BigQuery).
GraphQL	Lenguaje de consulta; respuesta JSON.	APIs con clientes que necesitan flexibilidad.

Comparativa XML vs JSON (la elección más frecuente en desarrollo web):

Tabla 2.5: XML vs JSON — comparación detallada.

Criterio	XML	JSON
Verbosidad	Alta (etiquetas de apertura y cierre)	Baja (sintaxis tipo objeto)
Tipos de datos	Solo strings, requiere XSD para tipado	Strings, números, booleanos, null, arrays, objetos
Validación	XSD, DTD, RelaxNG	JSON Schema
Soporte JavaScript	Requiere parsing manual	<code>JSON.parse()</code> nativo
Atributos	Sí (atributos en elementos)	No (todo son propiedades)
Comentarios	Sí (<code><!-- --></code>)	No (JSON5 permite)
Tamaño típico	100 % (referencia)	60–70 % del XML equivalente
Casos de uso 2026	Sistemas legados, configuración (Maven, Spring XML)	APIs REST, configuración (<code>package.json</code>)

Ejemplo comparativo: la misma información en cuatro formatos.

XML (Listado 2.10):

```
<usuario id="42">
  <nombre>Maria Lopez</nombre>
  <edad>23</edad>
  <activo>true</activo>
  <roles>
    <rol>USER</rol>
    <rol>EDITOR</rol>
  </roles>
</usuario>
```

Listing 2.10: Ejemplo de XML

JSON (Listado 2.11):

```
{  
  "id": 42,  
  "nombre": "Maria Lopez",  
  "edad": 23,  
  "activo": true,  
  "roles": ["USER", "EDITOR"]  
}
```

Listing 2.11: Ejemplo de JavaScript

YAML (Listado 2.12):

Listing 2.12: Ejemplo de Código

```
id: 42  
nombre: Maria Lopez  
edad: 23  
activo: true  
roles:  
  - USER  
  - EDITOR
```

TOML (Listado 2.13):

```
id = 42  
nombre = "Maria Lopez"  
edad = 23  
activo = true  
roles = ["USER", "EDITOR"]
```

Listing 2.13: Ejemplo de Shell

2.7 Buenas prácticas profesionales en HTML5

Conocer la sintaxis no basta para escribir HTML profesional: hay convenciones, criterios de mantenibilidad y restricciones de accesibilidad que distinguen el código de producción del código estudiantil. La caja siguiente compila quince reglas que el estudiante debe respetar al entregar HTML5 en cualquier proyecto.

Quince reglas profesionales de HTML5

1. Usa el doctype HTML5 simple `<!DOCTYPE html>`.
2. Define el charset al inicio del `<head>`, en los primeros 1024 bytes del archivo.
3. Define el viewport en toda página que pretenda ser mobile-friendly.
4. Cierra todas las etiquetas (incluso las que HTML5 permite no cerrar).
5. Usa comillas dobles en valores de atributos por consistencia.
6. No abuses de `<div>`: si existe un elemento semántico, úsalo.
7. Atributos booleanos sin valor.
8. Imágenes con alt siempre.
9. Imágenes con width y height para evitar *layout shift*.
10. Lazy loading en imágenes *below the fold*.
11. Formularios con `<label>` asociados.
12. Atributo autocomplete con valores estándares.
13. Headings sin saltos jerárquicos.

14. Validar en <https://validator.w3.org/nu/> antes de cada commit.
15. No uses CSS inline excepto en correos electrónicos.

2.8 Ejercicios

Los siguientes ejercicios consolidan los temas de la semana 2. El primero está completamente resuelto con código y captura de pantalla del resultado; el segundo es propuesto para que el estudiante lo desarrolle.

Ejercicio 2.1 (RESUELTO) — Formulario de registro completo y accesible

Objetivo: construir un formulario completo con ocho tipos diferentes de `<input>`, validación HTML5, accesibilidad y feedback visual.

Paso 1 — Crear proyecto en IntelliJ. New Project → HTML → Name: appweb-formularios. Crear registro.html.

Paso 2 — Código completo.

```

1 <!DOCTYPE html>
2 <html lang="es-EC">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1">
6   <title>Registro completo - UTEQ</title>
7   <style>
8     body { font-family: system-ui, sans-serif; max-width: 600px;
9           margin: 2rem auto; padding: 0 1rem; }
10    fieldset { margin-bottom: 1rem; padding: 1rem;
11             border: 1px solid #aaa; border-radius: 6px; }
12    legend { font-weight: bold; color: #006633; }
13    label { display: block; margin-top: .8rem; font-weight: 600; }
14    input, select, textarea {
15      width: 100%; padding: .5rem; box-sizing: border-box;
16      border: 1px solid #aaa; border-radius: 4px;
17      font-size: 1rem; margin-top: .25rem;
18    }
19    input:user-invalid { border-color: #b00020; }
20    input:user-valid { border-color: #2e7d32; }
21    small { display: block; color: #555; font-size: .85rem; }
22    button { margin-top: 1rem; padding: .7rem 1.2rem;
23           background: #006633; color: white; border: 0;
24           border-radius: 4px; font-size: 1rem; cursor: pointer; }
25  </style>
26 </head>
27 <body>
28 <main>
29   <h1>Registro de estudiantes UTEQ</h1>
30   <form action="/api/registro" method="POST">
31     <fieldset>
32       <legend>Datos personales</legend>
33
34       <label for="cedula">Cedula ecuatoriana:</label>
35       <input id="cedula" name="cedula" type="text"
36             pattern="[0-9]{10}" required>

```

```

37     <small>10 digitos sin guiones</small>
38
39     <label for="email">Correo institucional:</label>
40     <input id="email" name="email" type="email"
41           pattern=".+@uteq\.edu\.ec" required>
42
43     <label for="tel">Telefono celular:</label>
44     <input id="tel" name="tel" type="tel"
45           pattern="09[0-9]{8}" required>
46
47     <label for="fnac">Fecha de nacimiento:</label>
48     <input id="fnac" name="fnac" type="date"
49           min="1980-01-01" max="2010-12-31" required>
50
51     <label for="url">Sitio web (opcional):</label>
52     <input id="url" name="url" type="url">
53 </fieldset>
54
55 <fieldset>
56     <legend>Datos academicos</legend>
57
58     <label for="prom">Promedio (6.0-10.0):</label>
59     <input id="prom" name="prom" type="number"
60           min="6.0" max="10.0" step="0.01" required>
61
62     <label for="color">Color institucional preferido:</label>
63     <input id="color" name="color" type="color" value="#006633">
64
65     <label for="carrera">Carrera:</label>
66     <select id="carrera" name="carrera" required>
67         <option value="">-- Seleccione --</option>
68         <option value="SW">Ingenieria de Software</option>
69         <option value="SE">Ingenieria de Sistemas</option>
70     </select>
71 </fieldset>
72
73     <button type="submit">Registrarme</button>
74 </form>
75 </main>
76 </body>
77 </html>

```

Listing 2.14: registro.html

Resultado esperado en el navegador:

[Vista renderizada en Chrome]

Registro de estudiantes UTEQ

Datos personales

Cedula ecuatoriana:

10 dígitos sin guiones

Correo institucional:

Teléfono celular:

Fecha de nacimiento:

*(calendario nativo)**Datos académicos*

Promedio (6.0-10.0):

*(controles arriba/abajo)*Color preferido: *(selector)*Carrera:

Verificación de validaciones nativas:

1. Pulsar **Registrarme** con campos vacíos: el navegador detiene el envío y enfoca el primer campo inválido mostrando tooltip.
2. Escribir email con dominio diferente a **uteq.edu.ec**: el borde se vuelve rojo (gracias a `:user-invalid`).
3. En móvil (F12 → Toggle device toolbar): hacer clic en campo teléfono → aparece teclado numérico; en email → aparece tecla **@**; en fecha → aparece selector calendario.

Ejercicio 2.2 (PROPUESTO) — Página de portafolio profesional

Construir una página **portafolio.html** que demuestre dominio integral de HTML5 semántico. Requisitos:

- Estructura semántica completa: `<header>`, `<nav>`, `<main>`, `<article>` (al menos 3), `<aside>`, `<footer>`.
- Al menos un `<figure>`+`<figcaption>` con `<picture>` sirviendo múltiples resoluciones.
- Un `<video>` con al menos un `<track>` de subtítulos (crear archivo `.vtt` con 3-4 líneas).
- Un formulario de contacto con al menos 6 tipos diferentes de `<input>` HTML5: `text`, `email`, `tel`, `date`, `number`, `url`.
- Un elemento `<details>`+`<summary>` para una sección de preguntas frecuentes.
- Una tabla `<table>` con `<caption>`, `<thead>`, `<tbody>`, `<th scope>`.

Validar: cero errores en W3C, cero violaciones críticas en axe DevTools, navegación completa por teclado, y todos los tipos de input invocando teclado virtual correcto en móvil.

Entrega: archivo HTML + screenshots de validadores.